AsymptoticInverse

Prioritized implementation and validation plan

Vladimir Reshetnikov

September 2026

Abstract

This plan retains every authorized roadmap proposal and orders their implementation around shared mathematical contracts. The first stage extends the existing engines through explicit coordinate transformations. Later stages add exact-core perturbations, numerical certificates, larger logarithmic scales, composable result operations, incremental computation, finite exponential and Fourier sectors, and special-function adapters. Each stage has concrete acceptance cases and separate requirements for mathematical justification, exact symbolic checks, native tests, numerical evidence, performance measurements and rendered documentation. Three delivered milestones implement the nine stages for the admitted finite families. The acceptance register records their evidence and the remaining broader mathematical boundaries.

Contents

1 Scope, priorities and completion rules

The starting point is the reviewed power–log inverse package, including its Lambert-coordinate engine, exact finite-inverse certificates and sparse arithmetic improvements. The present campaign expands that foundation. Existing tests remain regression requirements, and the article's established distinction between the original function, an exact core, and a truncated forward model remains part of every result contract.

The work is organized into nine stages. A higher number indicates a dependency order, not permission to omit the stage. Independent implementation lanes may proceed concurrently when they do not edit the same files or compete for the native test runner.

Stage	Deliverable	Main prerequisite	Status
1	General exponential phases, pure logarithmic cores, and finite exponential sums through explicit coordinates	Existing inverse engines	Complete
2	Exact-core perturbations and transported input remainders	Coordinate contracts	Complete
3	Residual certificates and adaptive absolute/relative accuracy	Valid error transport	Complete
4	Reciprocal logarithms, iterated logarithms and nonpolynomial logarithmic coefficients	Scale representation	Complete
5	Composable arithmetic, inverse observables and refinement	Scale and provenance contracts	Complete
6	Incremental enumeration, Newton and grouped coefficient performance	Stable result semantics	Complete
7	Finite flat sectors and finite Fourier coefficient algebras	Extended scales and residual checks	Complete
8	Special-function and threshold adapters	Branch-aware coordinate dispatch	Complete
9	Generated independent tests and version-range validation	Continuous from stage 1	Complete

A stage is complete only when its accepted input family has a stated mathematical domain, its implementation returns the documented semantics, its required native tests pass, and its source documentation is updated. Claims about a PDF require rebuilding and inspecting the PDF. Claims about publication require a verified commit and remote reference. Mathematical proof, successful symbolic simplification, numerical agreement and passing tests are distinct forms of evidence and are reported separately.

Unsupported instances within a broader future family must be identified explicitly. A bounded implementation may support a mathematically defined subclass, but it must not silently relabel the whole roadmap family as completed. If a stage requires further work, its remaining cases and exact blockers stay in the plan.

First delivered milestone. Version 1.2.0 adds logarithmic target transformations, including signed amplitudes, target offsets, source translations, finite flat exponentials and both source infinities. It also adds persistent term-goal enumeration, exact-precision Newton states and an independent grouped Lagrange method. The eight delivered native suites pass 161 tests on Wolfram 15.0.1. Three reproducible performance comparisons have equal results; the term-goal

and Newton fixtures improve, while the small grouped fixture is slower. The article supplies the coordinate and grouped-coefficient derivations. The later milestones preserve the remaining families and acceptance obligations listed below.

Second delivered milestone. Version 1.3.0 adds source-logarithm and finite-exponential-sum charts with transported reconstruction errors, ten public calculus operations, exact-core marker corrections with a complete-tail theorem, and rational interval certificates with adaptive absolute and relative accuracy. Forty-six generated cases use independent inverse oracles. Certificates distinguish the stored explicit equation from a declared input remainder. Wolfram 15.0.1 is validated; the installed 14.3 engine cannot start because its license is invalid. The former untested 13.0 minimum is replaced by 15.0. The tests and artifact manifest for this milestone record the exact suite, source hashes, PDF builds and visual review.

General exponential exact-core perturbations, transformed input-error contracts, automatic discovery of certificate intervals, broader composite observables, public incremental refinement, and additional generated families remained open at that checkpoint. The reciprocal-log, flat-sector and other subsequent modules were not part of the second milestone. Automatic interval discovery remains an optional extension: Stage 3 requires and validates caller-supplied branch intervals.

Third delivered milestone. Version 1.4.0 implements the remaining finite logarithmic, flat-sector, Fourier and special-function families. Exact growing exponential cores retain their Lambert inverse while separate corrections resolve terms beyond every logarithmic order. General exact-core power observables, including negative powers, carry their own input-error ceilings. Public refinement retains coefficient states and polynomial powers, reports replay when a source is recomputed, and accepts additional-block or certified numerical-tolerance requests. Reciprocal-log calculus and flat-sector operations have separate proved error contracts.

The acceptance audit added the previously missing coordinate and calculus edge cases, tolerance sweeps, precision failure/recovery and independent multiscale comparisons. It found and corrected a certificate retry that could not repair bracket geometry, a lost negative Erfc reconstruction sign, and an affine Lambert threshold limit inconsistency. Seeded native marker oracles and a standalone evidence-preserving campaign complement the fixed regressions. Nine refinement benchmarks report exact equality, reused work, time, retained sizes and evaluation memory; some small fixtures remain slower despite eliminating repeated coefficient work.

The manifest `validation/milestone-3-artifacts.json` records the native suite, benchmark and generated-campaign evidence, source hashes, strict PDF builds and actual rendered-page review. Complete status in the register refers to the accepted families and contracts below, including explicit rejection boundaries; it does not claim a general transseries field or validation on an unlicensed or untested Wolfram version.

1.1 Acceptance register and continuing boundaries

1. **Coordinates.** Target-log and source charts retain the original equation, side and reconstruction. Named quadratic-logarithm, signed, shifted, cancelled-rate and incommensurable-rate cases have independent formulas and numerical checks at several scales in `Tests/AcceptanceCoordinatesCalculus.wlt` and the coordinate suites. More general source transformations require an invertible chart and its own error-transport proof.

2. **Exact cores and input errors.** Core and exponential-core suites cover the four named equations, signed/scaled branches, finite reconstruction and negative observables. A declared error includes its matching derivative hypothesis; it limits the returned accuracy. Arbitrary

exact cores and unrelated exponential phases require new recognizers, inverse identities and tail proofs.

3. **Certificates.** Exact rational intervals certify continuity, existence and a separated derivative. Three absolute and three relative tolerances across each of the six named families, plus threshold, insufficient-precision and invalid-bracket cases, are covered by `Tests/AcceptanceCertificates.wlt`. The certified equation is the stored explicit function, not an unspecified input-error family. Automatic interval discovery is additional work, not an accepted input precondition removed by this release.

4. **Logarithmic scales.** Reciprocal rational units, generalized Laurent coefficients and leading monomials in finitely many positive logarithmic levels have closure and tail theorems. The logarithmic and reciprocal-operation suites cover the named examples, two cutoffs, composition and qualified derivatives. Arbitrary sums of incomparable leading coefficients and simultaneous independent cutoffs at every level remain outside these finite algebras.

5. **Calculus and refinement.** Boundary logarithmic degrees, uncertain reciprocals, paired inverse observables, vanishing outer derivatives and translated endpoints have explicit tests. Refinement reports state reuse or source replay, stops at exact finite termination, and preserves the best expansion when a resource bound intervenes. Operations across incompatible coordinates return a failure until a proved conversion is available.

6. **Computation.** Retained complete weight layers, Newton precisions and bounded polynomial-power caches preserve exact results and resource budgets. The nine-case benchmark includes every named parameter family and deep one-gap Catalan coefficients through cutoff 64. It records regressions as well as improvements; no portable time threshold defines correctness.

7. **Flat and Fourier sectors.** Flat inverses retain an exact monomial zero sector, complete commensurable exponential sectors, separate inner cutoffs, polynomial observables, products and qualified derivatives. The flat/Fourier suites include independent residuals, multiscale numerics, resonance and budget failures. Incomparable exponential phases, general nonlinear sector composition and sign-changing leading Fourier blocks remain explicit boundaries.

8. **Special functions.** Erfc, Gamma and LogGamma tails carry forward value/derivative error contracts. Lambert and quadratic threshold adapters retain their real ramified branch, including signed affine targets. Independent original-function, adjacent-model and threshold overlap tests validate these explicit adapters. No unproved automatic switching point or uniform critical-parameter theorem is asserted.

9. **Generated validation and versions.** Independent radical and native marker-forward oracles cover the specified signs, gaps, collisions, logarithms, directions, observables, remainders and resources. The standalone runner saves seeds, full inputs, hashes, messages and all completed shrink attempts. Native compatibility is established for Wolfram 15.0.1 only. The installed 14.3 kernel cannot start; the minimum version constraint and documentation explicitly exclude a claim for it.

2 Shared contracts

2.1 Coordinates and branch provenance

Every transformed inverse retains the original forward function, the source endpoint and side, the transformation, its inverse reconstruction, the transformed target, and the assumptions

under which these maps are real and locally one-to-one. The actual expansion variable and cutoff meaning must be inspectable. A cutoff in $1/\log y$ must never be presented as a cutoff in $1/y$.

If $t = T(x)$ is a source coordinate and $x = H(t)$ its reconstruction, solve $F(H(t)) = y$ in the t coordinate, then transport the approximation and its error through H. A target transformation $v = J(y)$ instead solves $J(F(x)) = v$ on a domain where J is one-to-one. Neither transformation changes which source branch the user requested.

2.2 Remainder information

An expansion is a finite expression together with a scale, a remainder assertion and its hypotheses. Fixed asymptotic constants are not numerical error bounds until they have been bounded explicitly. An external prefactor multiplies the remainder as well as the displayed expression. Differentiating an unknown remainder requires derivative control; it is not inferred from a value bound alone.

For reconstruction $x = e^{st}$, $s \neq 0$, an additive approximation $t = A + E$ gives a relative approximation $x = e^{sA}(1 + \mathcal{O}(E))$ only when $E \to 0$. If E does not tend to zero, the implementation must refine the inner expansion or preserve the exponential enclosure explicitly. It must not exponentiate a coarse additive expansion and claim a small relative error.

2.3 Exactness and refinement

The result separately records whether the original forward model is exact, whether the inverse expression is exactly certified, and whether a displayed core omits smaller sectors. A zero remainder needs an exact argument about the original function on the selected branch. Refinement preserves the original assumptions, domain and provenance, and states whether it reused prior coefficients or recomputed them.

3 Stage 1: coordinate transformations

Priority and outcome. This is the first implementation milestone. It removes input restrictions caused by the current recognizers while reusing the established power–log and Lambert engines.

3.1 General exponential phases

Recognize $F(x) = a\exp(\Phi(x))$, and products that can be brought into that form on a proved positive domain. Solve $\Phi(x) = \log(y/a)$. Include phases with several distinct powers, polynomial–log terms, affine shifts, and negative signs when the target approaches a finite limit. The transformed inverse carries its own residual; the result also exposes the original residual or a rigorously equivalent transformed residual.

Acceptance cases include:

- e^{x^2+x} at $+\infty$, checked against $(-1 + \sqrt{1 + 4\log y})/2$;
- $e^{x^2+x\log x}$ and $x^3e^{x^2+x}$ at $+\infty$, with independent phase residuals;
- $e^{(x-3)^2+(x-3)}$ with the requested source endpoint, so translation cannot disappear;
- e^{-x^2-x} at $+\infty$, with $y \to 0^+$ and the same quadratic inverse after $\log y \mapsto -\log y$;

- sign-scaled and target-offset versions such as $7 - 2e^{x^2+x}$, with the actual target side checked.

The forward phase must be selected structurally or proved equivalent under assumptions. An unproved logarithm identity involving a negative factor is a failure, not a simplification rule.

3.2 Pure logarithmic cores

Use $t = \log x$ at $x \to +\infty$ or $t = -\log u$ at a positive source distance $u \to 0$. Invert the resulting power–log function of t and reconstruct the original source variable. General powers of logarithmic expressions and finite logarithmic polynomials are included when the transformed branch is valid.

Acceptance cases are $\log x$, $(\log x)^2 + \log x$ at $+\infty$, $-\log x$ at 0^+, $(-\log x)^2 - \log x$ at 0^+, and shifted signed source distances. The quadratic case has an exact exponential reconstruction and tests whether the inner error is refined until it tends to zero. A coarse term count must not create an invalid relative exponential remainder.

3.3 Finite exponential sums

For finite sums $\sum_j a_j e^{\lambda_j x}$ with proved real rates and a selected endpoint, use a positive exponential source coordinate. Rates may be rationally related or incommensurable; the resulting powers are handled by the exact exponent engine. A common nonzero rate is chosen with a sign that makes the source coordinate tend to zero. A nonconstant limit or cancellation is resolved before selecting the leading rate.

Acceptance cases are $e^x + e^{2x}$, $e^x + e^{\sqrt{2}x}$, $e^{-x} + 2e^{-3x}$ at $+\infty$, and $3 - e^{-x} + e^{-2x}$. The first has the exact oracle $x = \log((\sqrt{1+4y} - 1)/2)$. Include cancellation and duplicate-rate cases, signed target limits, and both source infinities.

Acceptance evidence. Each family needs exact transformed composition, an oracle where available, high-precision comparisons at several scales, domain-rejection tests, and observable/remainder tests. The article must state the coordinate error-transport theorem and the actual cutoff semantics before these families are described as supported.

4 Stage 2: exact cores, perturbations and input errors

Introduce a structured exact-core interface carrying a forward core F_0, its inverse ϕ, the branch domain, the perturbation and any declared remainder. The existing marker formula generates coefficients, but its output must gain scale and error metadata. A core identity is either structurally known or separately certified.

Support finite perturbation orders around Lambert and coordinate-transformed cores. Preserve both perturbation order and logarithmic order when they are independent; corrections beyond every logarithmic order must remain visible as unresolved sectors until this stage computes them. Generalized input remainder declarations specify the coordinate, whether the bound is additive or relative, and the derivative information needed for inversion.

Acceptance cases include $x \log x + x^2$ near 0^+, $xe^x + x^2$ at $+\infty$, a shifted/scaled version of each, and $x + \log x + x^{-1}$ at infinity. Compare an exact-core perturbative correction against an independently computed residual and numerical difference from the core inverse. Declared remainders must cap both ordinary and transformed inverse observables correctly, including negative powers and finite-source reconstruction.

This stage must also reject an input remainder that destroys local monotonicity or lacks the derivative/branch contract needed to justify transport. A conditionally valid formal correction is labelled as such rather than returned as an unconditional asymptotic statement.

5 Stage 3: certificates and adaptive accuracy

5.1 Residual certificates

Add an explicit certification operation for a numerical target. Given an approximation A and a proved nonzero slope bound $|F'| \geq \mu$ on a branch interval, derive an enclosure from the residual. Existence requires a bracket contained in that interval; uniqueness requires the derivative sign. An upper slope bound supplies the corresponding lower error estimate when useful.

The certificate records the target, evaluation precision, interval, residual enclosure, slope enclosure, proved source side, resulting inverse interval and method. Interval or directed-rounding calculations must be used when the certificate relies on numerical bounds. A successful `FindRoot` call alone is not a certificate.

Acceptance cases include the globally monotone $x + x^2(1 + \log x)$, $x + x^{\sqrt{2}}$, a nonunit leading power, both Lambert branches and a transformed exponential phase. Include a target near a branch threshold, a deliberately invalid bracket and insufficient numerical precision.

5.2 Adaptive absolute and relative accuracy

Implement requested absolute or relative accuracy by increasing expansion order and working precision until a certificate meets the tolerance. Near a zero of the inverse, a relative request needs an explicit absolute fallback or a nonzero lower bound. For convergent expansions, use a valid tail majorant where available; for asymptotic-only families, detect ineffective additional terms and report the unresolved error floor.

Acceptance requires several tolerances per family, observed refinement histories, verified stopping conditions, and a bounded failure mode. An approximation based only on a core cannot claim a tolerance smaller than its unresolved perturbation sector. Resource exhaustion preserves the best available approximation and evidence without claiming the requested accuracy was achieved.

6 Stage 4: reciprocal and iterated logarithms

6.1 Reciprocal-logarithm valuation

Extend the scale model to allow a logarithmic coordinate independent of the ordinary power coordinate. This directly covers zero-gap corrections such as $x + x/\log x$. Define an exclusive logarithmic cutoff, degree/valuation conventions for reciprocal powers, composition and differentiation, and an ordering policy for mixed power and logarithmic terms.

Acceptance includes the convergent expansion

$$(x + x/\log x)^{-1}(y) = y\left(1 - \frac{1}{\log y} + \frac{1}{\log^2 y} - \frac{2}{\log^3 y} + \frac{7}{2\log^4 y} + \cdots\right),$$

with the inverse notation understood as function inversion. Add mixed inputs such as $x(1 + 1/\log x) + x^2$ and a case whose first logarithmic correction cancels. The result must distinguish an omitted reciprocal-logarithm block from an exponentially smaller power block.

6.2 Iterated logarithms and nonpolynomial coefficients

Support finitely many explicitly declared logarithmic levels, including $\log(-\log x)$ and real powers of $-\log x$ on $x \to 0^+$. The coefficient algebra must be closed under the derivatives actually used; for example $(-L)^{1/2}$ requires negative half-integer powers after differentiation.

Acceptance cases are $x + x^2\sqrt{-\log x}$, $x + x^2 \log(-\log x)$ and a leading core containing an iterated logarithm such as $x \log\log x$ at $+\infty$. Include sign-domain failures and two logarithmic truncation depths. This stage needs closure and remainder theorems for the supported finite hierarchy, not only parser changes.

7 Stage 5: composable results and observables

Provide public operations for addition, multiplication, division, integer and real powers, logarithms, allowed analytic composition, differentiation where justified, inverse observables and refinement. These operations retain errors and provenance. Incompatible coordinates either undergo a proved conversion or produce a descriptive failure. Computing with `Normal` remains an explicit operation that discards error metadata.

Acceptance cases include products where a discarded boundary block raises the logarithmic degree; division by a series with an uncertain leading term; two inverse observables of the same branch; composition with an analytic function whose first derivatives vanish; and a shifted finite endpoint. The equality of coefficients produced directly and through result operations is checked independently of their remainder assertions.

Refinement requests a larger cutoff, additional blocks or a numerical tolerance while preserving branch assumptions. It must handle a terminating exact inverse without waiting forever for nonexistent terms. Cached intermediate objects are tied to their original data and assumptions, and a result reports what was reused.

8 Stage 6: incremental and grouped computation

Reuse the retained multi-index region, its frontier, polynomial powers and coefficient blocks when increasing a cutoff. Group equal exponent weights before expensive canonicalization where possible. Add a grouped coefficient route or dynamic program for repeated gaps and resonances, and use Newton iteration where it reduces the measured work.

The relevant complexity parameters are retained weights, contributing multi-indices, polynomial degrees, coefficient-field cost and the requested precision. A logarithmic Newton iteration count is not by itself a runtime guarantee. Keep the existing exact Lagrange route as an independent reference for bounded cases.

Benchmark acceptance includes dense rational gaps, two and three irrational gaps, repeated/colliding weights, high logarithmic degrees, deep one-gap expansions, and successive refinement requests. Measure total elapsed time, retained result size and bounded allocation or peak memory where available. Every benchmark verifies equality of the compared results. Performance regressions are reported alongside improvements; absolute wall-time thresholds are not used as portable correctness tests.

Resource budgets must cover incremental state as well as output. The grouped and Newton engines must not produce a weaker remainder merely to obtain a faster benchmark.

9 Stage 7: finite flat sectors and Fourier coefficients

9.1 Finite exponential sectors

Add a finite, explicitly truncated sector representation for positive flat scales such as $E = e^{-1/y}$. It needs a sector order, an inner power–log cutoff, multiplication and composition rules, derivative effects and a remainder that identifies both omitted levels. Begin with a mathematically defined finite family; do not claim a general transseries engine.

The primary acceptance case is the inverse of $x + e^{-1/x}$ at 0^+, whose first sectors are

$$y - E + y^{-2}E^2 + \left(y^{-3} - \frac{3}{2}y^{-4}\right)E^3, \qquad E = e^{-1/y}.$$

Further cases include a polynomial prefactor multiplying the flat term, two commensurable sectors, and an input whose sector hierarchy cannot be ordered. Validate the retained sectors by an independent residual calculation and sufficiently high precision to resolve their small differences; ordinary machine arithmetic is inadequate here.

9.2 Finite Fourier coefficient algebra

Represent finite sums of Fourier modes in $\log x$, with exact real frequencies when their equalities are provable. Differentiation preserves frequencies and multiplication convolves them; truncation in the power variable is separate from the finite set of frequencies. Remainder bounds use coefficient norms and derivative estimates instead of polynomial logarithmic degree alone.

Acceptance includes $x + x^2 \sin(\log x)$, whose inverse begins

$$y - y^2 \sin L + y^3 \sin L(2 \sin L + \cos L) + \mathcal{O}(y^4), \qquad L = \log y,$$

and a two-frequency case with resonance. A fixed frequency budget has an explicit failure or truncation policy. Oscillation does not justify assuming a sign for a leading coefficient that crosses zero.

10 Stage 8: special functions and thresholds

Create adapters as proved or documented reductions to the shared engines, with domain and branch information attached. Prioritize special functions already expressible through the available power–log, Lambert or transformed coordinates; then add asymptotic forward models with explicit error contracts.

Acceptance includes an inverse based on a valid Stirling-type model, a complementary-error-function tail, Lambert behavior near its real branch threshold, and a coalescing quadratic inverse branch. Near a finite critical point use the appropriate ramified local coordinate, not a large-argument Lambert expansion. For every adapter, identify the reference theorem or exact identity, its remainder conditions and the parameter range supported.

Threshold selection needs overlap tests between neighboring representations and must retain the user's requested branch through the transition. Numerical stabilization, such as avoiding overflow in $\log(e^y)$, is validated separately from the asymptotic formula. If a useful overlap or rigorous error estimate has not been established, the adapter exposes that limitation rather than silently selecting an unverified threshold.

11 Stage 9: independent generated tests and versions

Generated tests run throughout the campaign. They use bounded input families and a recorded seed. Their expected values come from exact inverse oracles, independent coefficient recurrences, independent symbolic substitution, interval residual certificates or high-precision solves as appropriate. Comparing two calls that share the same coefficient path is supplementary evidence, not the only oracle.

The generated families cover leading signs and powers, both endpoint directions, exact rational and irrational gaps, logarithmic degrees, collisions, cancellation, coordinate transforms, observable powers, cutoff boundaries, declared remainders and resource limits. Metamorphic checks include consistent rescaling/translation, refinement preserving earlier complete blocks, and composition of certified exact inverse pairs.

Every failure saves a compact reproducible input, assumptions, version, requested operation and observed result. Shrink failures toward small counterexamples. Tests that assert rejection must state which mathematical or resource contract is violated.

Version-range validation records the actual Wolfram kernel versions available and runs the common suite on each. Parser or `Series` differences are isolated in adapters or explicit supported-version constraints. One successful version does not establish an untested version range. Publish the tested range only after those native runs occur.

12 Integration, documentation and publication

One owner runs the native Wolfram tests and one owner performs shared-worktree Git mutations. Agents own disjoint implementation or documentation files, exchange mathematical contracts early, and avoid duplicating expensive test runs. Stage interfaces are reviewed before integration; they are not frozen prematurely when a counterexample reveals a missing invariant.

Each meaningful milestone includes a detailed commit stating the behavior changed, its rationale, mathematical/API effects, exact validation performed and remaining scope. Push only after verifying the intended staged files and the remote destination; record the resulting commit and remote state. Do not stage unrelated files or generated scratch artifacts.

Update the article with theorems, worked examples, actual APIs and precisely described limitations. Keep this plan as a status ledger: for every stage, replace Plannedonly after recording the implementation paths and evidence, and retain any remaining subfamily explicitly. Source and rendered PDF are delivered together. Every changed TeX document receives three serial strict `pdflatex` passes, followed by page rendering and actual layout review. An unbuilt source change must not be accompanied by a claim that the existing PDF represents it.

Completion requires all nine stages' acceptance evidence, passing regressions, precise remaining boundaries, inspected source/PDF pairs, and verified publication. Partially completed stages remain open.